

Object Oriented Programming & Java

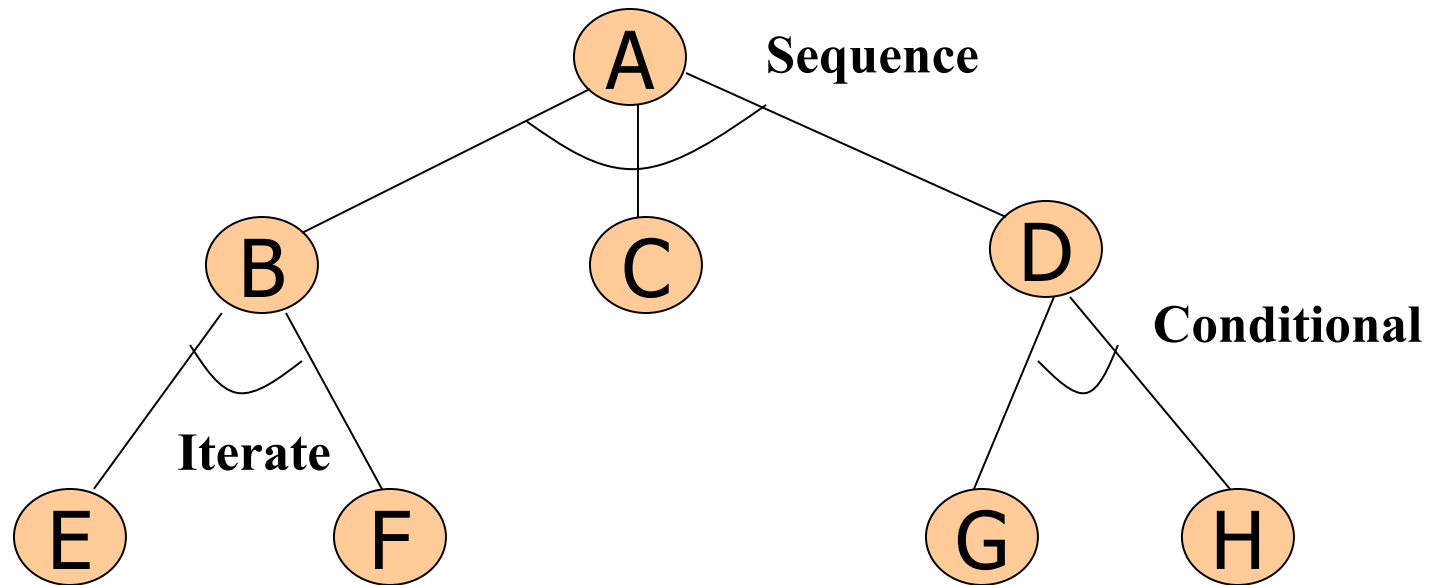
Pedro García López

pgarcia@etse.urv.es/

Copyright

- © University Rovira i Virgili
- Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.1 or any later version published by the Free Software Foundation; A copy of the license is available at:
 - <http://www.fsf.org/licenses/fdl.html>

Functional Decomposition



Problems

- Main function: “System root”
- Focused on the user interface
- Early temporal ordering
- Precludes software reuse
- Little attention paid to data structures
- As the system changes, data is more stable than functions

Object Oriented Design

- Objects are more stable when the system changes (*Extensibility*).
- Data types and operations (*Reuse*).
- Designer list operations associated to a class.
- Delay the order of execution of functions.

OOP Design

1. Identify Classes (Entities)
2. Find class operations and properties
3. Describe object types
4. Find class relationships
5. Use objects to structure software

Class relationships

•Composition (HAS a/N)

A class A is client of class B , if A contains an entity e of type B ($e:B$)



•Inheritance (IS a)

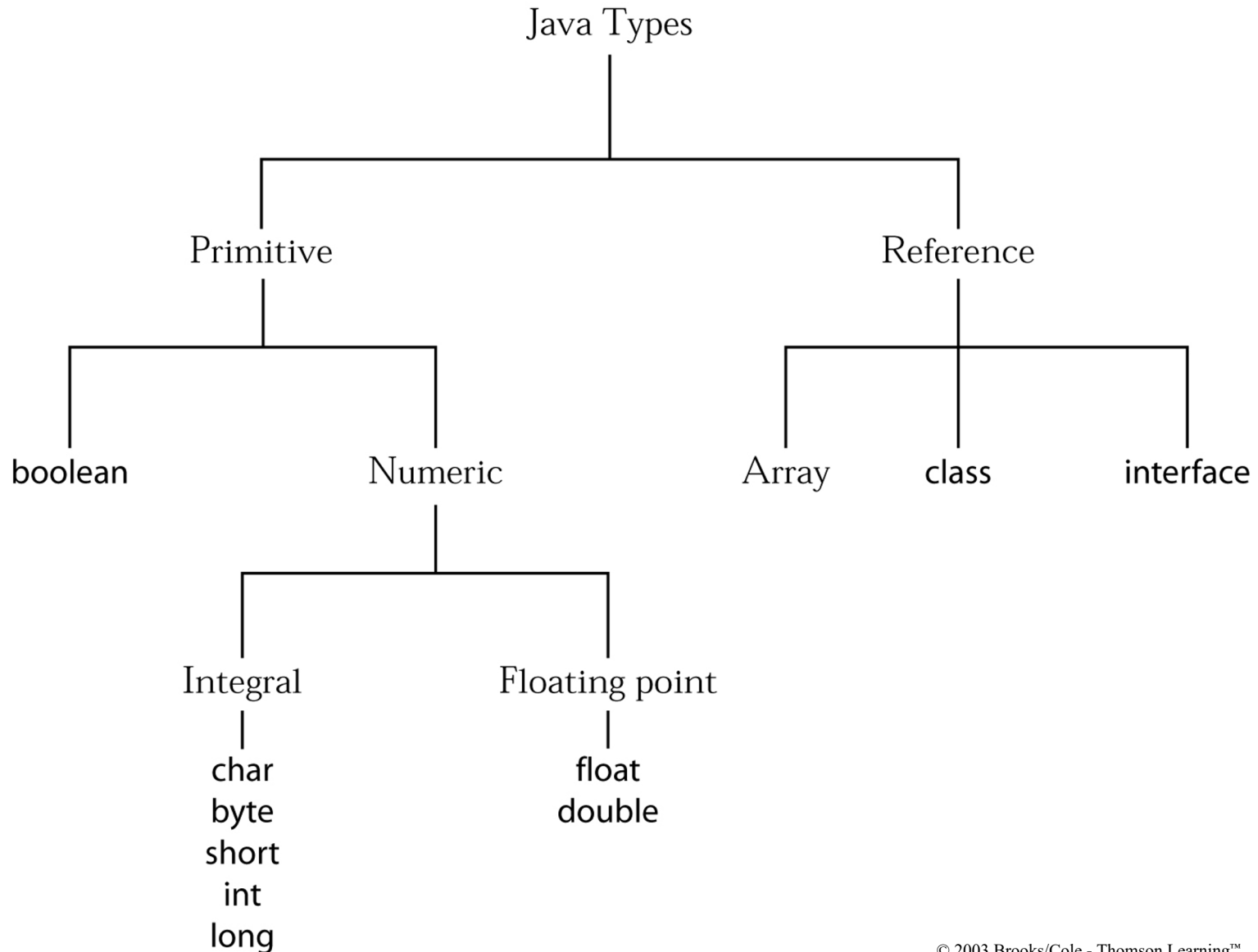
Specialized version



Key concepts

- Class and Object
- Class: Attributes and methods
- Class creation: constructors and instantiation
- Methods invocation
- Class & Instance Attributes and methods
- Information hiding & Encapsulation
- Reference types, assignment, aliasing, cloning
- MetaClass
- Class Relationships: composition and inheritance
- Inheritance types
- Subtype Polymorphism and dynamic binding

Type system



Simple types

Tipos Simples	Tamaño	Valor Mínimo	Valor Máximo	Tipos Wrapper
boolean	1-bit	-	-	Boolean
char	16-bit	Unicode 0	Unicode $2^{16}-1$	Character
byte	8-bit	-128	+127	Byte
short	16-bit	-2^{15}	$+2^{15}-1$	Short
int	32-bit	-2^{31}	$+2^{31}-1$	Integer
long	64-bit	-2^{63}	$+2^{63}-1$	Long
float	32-bit	IEEE754	IEEE754	Float
double	64-bit	IEEE754	IEEE754	Double
void	-	-	-	Void

Primitive and wrapper types

- Wrapper types for every primitive type (Integer, Float, Char, Double)

```
int a =1;  
int b= 2*3;  
Integer awrap = new Integer(a);  
int x = awrap.intValue();
```

Auto boxing/unboxing

- Java 1.4 does not permit primitive types where you expect Objects (Wrapper)

-`myVector.add(new Integer(5));`

- You cannot use objects where you expect primitive types:

-`int n = ((Integer)myVector.lastElement()).intValue();`

- Java 5:

-`Vector<Integer> myVector = new Vector<Integer>();`
`myVector.add(5);`
`int n = myVector.lastElement();`

Class properties

- We cannot access from static methods to non static attributes

// Método de clase

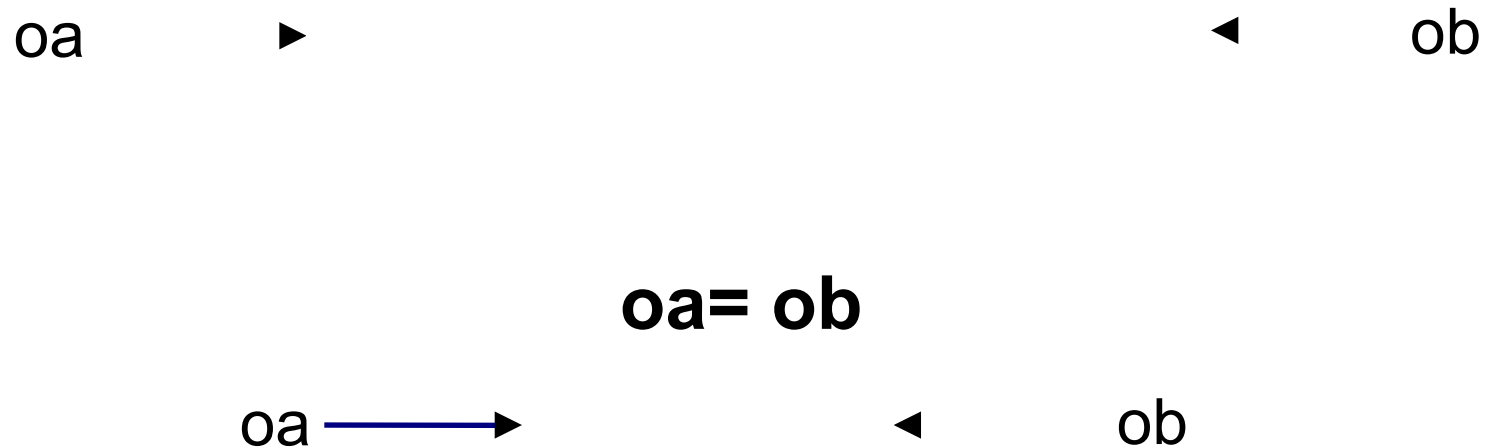
```
public static int setInterest(int amount)
{
    interest= amount
}
```

Class attributes

```
public class HelloWorld {  
    public static void main(String[] args) {  
        Account.setInterest(12);  
        Math.sqrt(9);  
    }  
}
```

Class attributes are valid for all instances

Aliasing and references



Assignment and cloning

```
Point p = new Point (1,1);
```

```
Point p2 = p;
```

```
p2.setLocation(120,120);
```

```
//Aliasing
```

```
System.out.println("Valor de p:" + p.getX());
```

```
System.out.println("Valor de p2:" + p2.getX());
```

```
Point p3 = (Point)p.clone();
```

```
p3.setLocation(40,40);
```

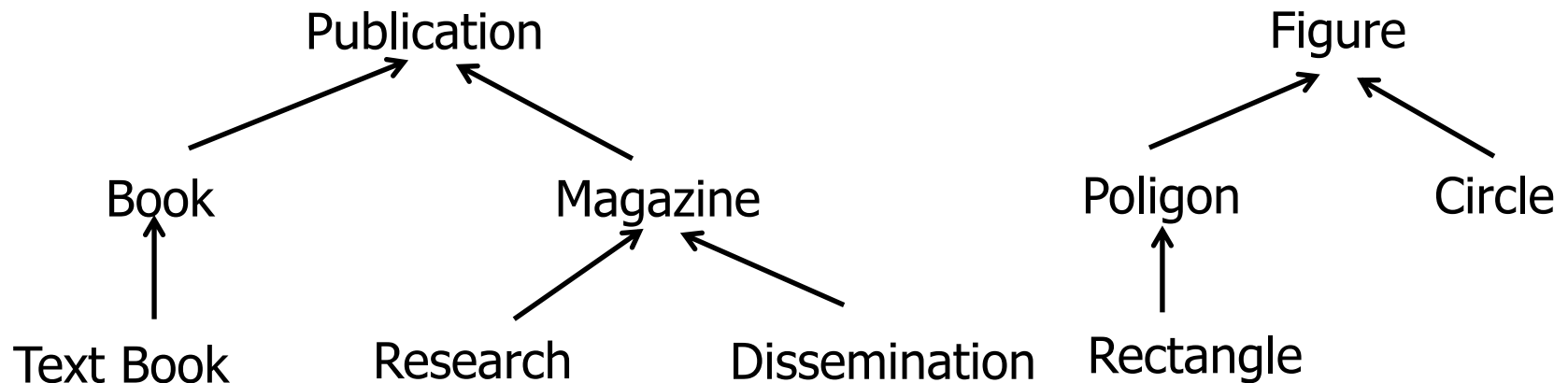
```
System.out.println("Valor de p:" + p.getX());
```

```
System.out.println("Valor de p3:" + p3.getX());
```

Inheritance

Class hierarchy

Examples:



- Code reuse
- Consistent with type system

Inheritance

If B inherits from A then B includes **structure** (attributes) and **behaviout** (methods) of class A , but it can be adapted;

- B can **add** new **attrs**
 - B can **add** new **methods**
 - B can **redefine** **methods**
 - B can **rename** attrs or methods
 - Abstract inheritance: B can implement a deferred method
 - Multiple inheritance: extending from two classes
- | | |
|---|---|
| { | <ul style="list-style-type: none">•<u>Refine</u>: Extend•<u>Replace</u>: Improve or modify |
|---|---|

Subtype polymorphism

- **Static type:**

- Associated to declaration (Animal a)

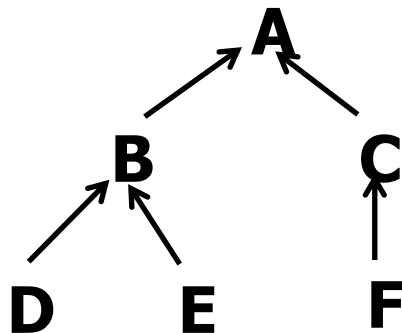
- **Dynamic type:**

- Connected in run time to that variable

- **Set of dynamic types:**

- Set of potential dynamic types

Example:



oa A; ob: B; oc; C; oa = oc;

st(oa) = A

{A,B,C,D,E,F}

st(ob) = B

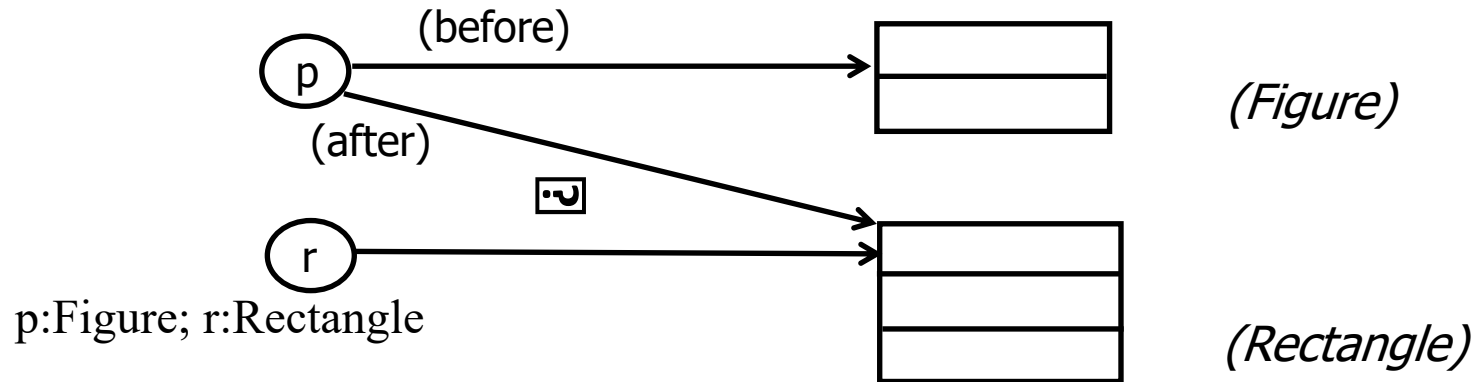
st(oc) = C

dt(oa) = C

sdt(ob) = {B, D, E}

sdt(oc) = {C,F}

Polymorphic connection



a) Polymorphic assignment:

$p = r$

-- *p* is a polymorphic entity

do(*r*)

b) Parameter passing

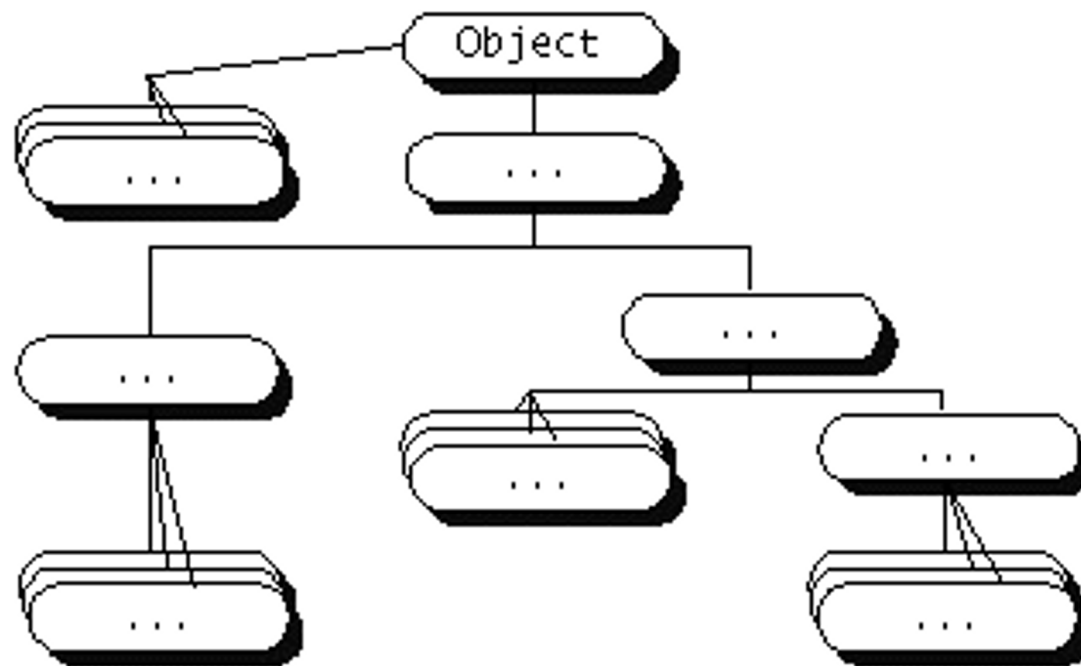
do (Figure *p* { ... }) – ***p* accepts all inheritance tree**

c) Dynamic binding

p.draw() – invokes the method in the dynamic type
(Rectangle)

Inheritance in Java

- **public class X extends Y {...}**



Syntaxis

```
[ ClassModifiers ] class ClassName  
  [ extends SuperClass ]  
  [ implements Interface1, Interface2 ... ] {  
    ClassMemberDeclarations  
  }
```

- Object class
- this and super

Object root class

protected Object clone()

Creates and returns a copy of this object.

boolean equals(Object obj)

Indicates whether some other object is "equal to" this one.
protected

void finalize()

Called by the garbage collector on an object when garbage collection determines that there are no more references to the object.

Class getClass()

Returns the runtime class of an object

String toString()

Returns a string representation of the object.

Object

- All classes inherit from Object by default
 - Class Account [extends Object]
- You can redefine methods equals, clone y toString.
- Take care when comparing Objects. Unlike primitive types, == will not work.

```
int x = 3;
```

```
int y = 5;
```

```
if (x == y) then (...)
```

```
Cuenta c = new Cuenta("pedro");
```

```
Cuenta d = new Cuenta("pepe);
```

```
if (c == d) then (...) NO !!!!
```

Subtype polymorphism

```
public class UsaAnimal
{
    public static void main(String[] args) {

        Animal a = new Animal(3,3);
        Perro p = new Perro(1,1,true);
        Gato g = new Gato(1,2);
        Loro l = new Loro(1,2);
        LoroMutante lM = new LoroMutante(3,3);

        System.out.println("ANIMAL:" + a.habla());
        System.out.println("PERRO:" + p.habla());
        System.out.println("GATO:" + g.habla());
        System.out.println("LORO:" + l.habla());
        System.out.println("LORO MUTANTE:" + lM.habla());

        // Asignaciones polimórficas y Ligadura dinámica
        a = g;
        System.out.println("ANIMAL:" + a.habla());

        a = lM;
        System.out.println("ANIMAL:" + a.habla());

        //System.out.println(a.vuela()); //ERROR

        LoroMutante otro = (LoroMutante) a;
        System.out.println(otro.vuela());

        // Asignación polimórfica
        Animal prueba = new Perro(1,10,true);
    }
}
```

Subtype polymorphism

```
public class UsaAnimal2
{
    public static void main(String[] args) {

        Animal a = new Animal(3,3);
        Gato g = new Gato(1,2);
        LoroMutante lM = new LoroMutante(3,3);

        // Estructura polimórfica
        Animal[] granja = new Animal[3];
        granja[0] = a;
        granja[1] = g;
        granja[2] = lM;

        // Ligadura dinámica
        for (int i=0; i<granja.length; i++)
            System.out.println(granja[i].habla());

        // Estructura polimórfica
        Object[] camion = new Object[3];
        camion[0] = a;
        camion[1] = g;
        camion[2] = lM;

        // Ligadura dinámica
        for (int i=0; i<camion.length; i++) {
            Animal x = (Animal) camion[i];
            System.out.println(x.habla());
        }
    }
}
```

Subtype polymorphism

```
Animal a = new Animal();
```

```
Perro p;
```

```
a = p;
```

```
a.habla()
```

```
Object x;
```

```
Perro p = new Perro();
```

```
x = p;
```

```
Perro p2 = (Perro) x;
```

Castings

```
Object list [] = new Object[3];
```

```
String s = “lala”;
```

```
Perro p = new Perro();
```

```
list[0] = s;
```

```
list[1] = p;
```

```
Object x = list[0];
```

```
Perro p2 = (Perro) list[1];
```

Assignments

```
Object list [] = new Object[3];
```

```
Integer i1 = new Integer(3);
```

```
int z = 74;
```

```
Integer i2 = new Integer(4);
```

```
list[0] = i1;
```

```
list[1] = i2;
```

```
Object x = list[0];
```

```
Integer r2 = (Integer) list[1];
```

```
int resul = z2.intValue();
```

Interfaces

An interface is different from a class in several ways, including:

We cannot instantiate an interface.

An interface does not contain any constructors.

All of the methods in an interface are abstract.

An interface is not extended by a class; it is implemented by a class.

Interfaces



interface Persona

```
{  
public void habla();  
}
```

interface Perro

```
{  
public void ladra();  
}
```

Interfaces and abstract classes

Abstract class	Interface
1) Abstract class can have abstract and non-abstract methods.	Interface can have only abstract methods. Since Java 8, it can have default and static methods also.
2) Abstract class doesn't support multiple inheritance .	Interface supports multiple inheritance .
3) Abstract class can have final, non-final, static and non-static variables .	Interface has only static and final variables .
4) Abstract class can provide the implementation of interface .	Interface can't provide the implementation of abstract class .
5) The abstract keyword is used to declare abstract class.	The interface keyword is used to declare interface.
6) Example: <pre>public abstract class Shape{ public abstract void draw(); }</pre>	Example: <pre>public interface Drawable{ void draw(); }</pre>

Interfaces and inheritance

```
public class Mutante implements Persona, Perro
{
    public void habla()
    {
        System.out.println("Soy una persona");
    }

    public void ladra()
    {
        System.out.println("GUAU");
    }
}
```

Interfaces and polymorphism

```
Mutante m = new Mutante();
```

```
Persona p = m;
```

```
p.habla();
```

```
Perro p = m;
```

```
m.ladra();
```

Interfaces

```
class Prueba  
{  
    public static void ladra (Perro p)  
    {    p.ladra();  
    }  
    public static void habla(Persona p)  
    { p.habra()  
    }  
}
```

Interfaces

```
Mutante m = new Mutante();
```

```
Prueba.habla(m);
```

```
Prueba.ladra(m);
```

We cannot instantiate interfaces, but we can assign them classes implementing one of them.

```
Persona p = new Mutante();
```

Interfaces

```
Mutante m = new Mutante();
```

```
Prueba.habla(m);
```

```
Prueba.ladra(m);
```

We cannot instantiate interfaces, but we can assign them classes implementing one of them.

```
Persona p = new Mutante();
```

Default methods and Fragile Base Class Problem

The **fragile base class problem** is a fundamental architectural problem of [object-oriented programming](#) systems where base classes ([superclasses](#)) are considered "fragile" because seemingly safe modifications to a base class, when inherited by the [derived classes](#), may cause the derived classes to malfunction. (Wikipedia)

```
interface Shape
{
    // String color{ } //error, since method not declared default
    default void color(){
        System.out.println("Red");
    }
    double area();
}

class Circle implements Shape
{
    double radius;
    public double area(){
        return 3.14*radius*radius;
    }
}

public class Test
{
    public static void main(String args[]){
        Circle c = new Circle();
        System.out.println(c.area());
        c.color();
    }
}
```

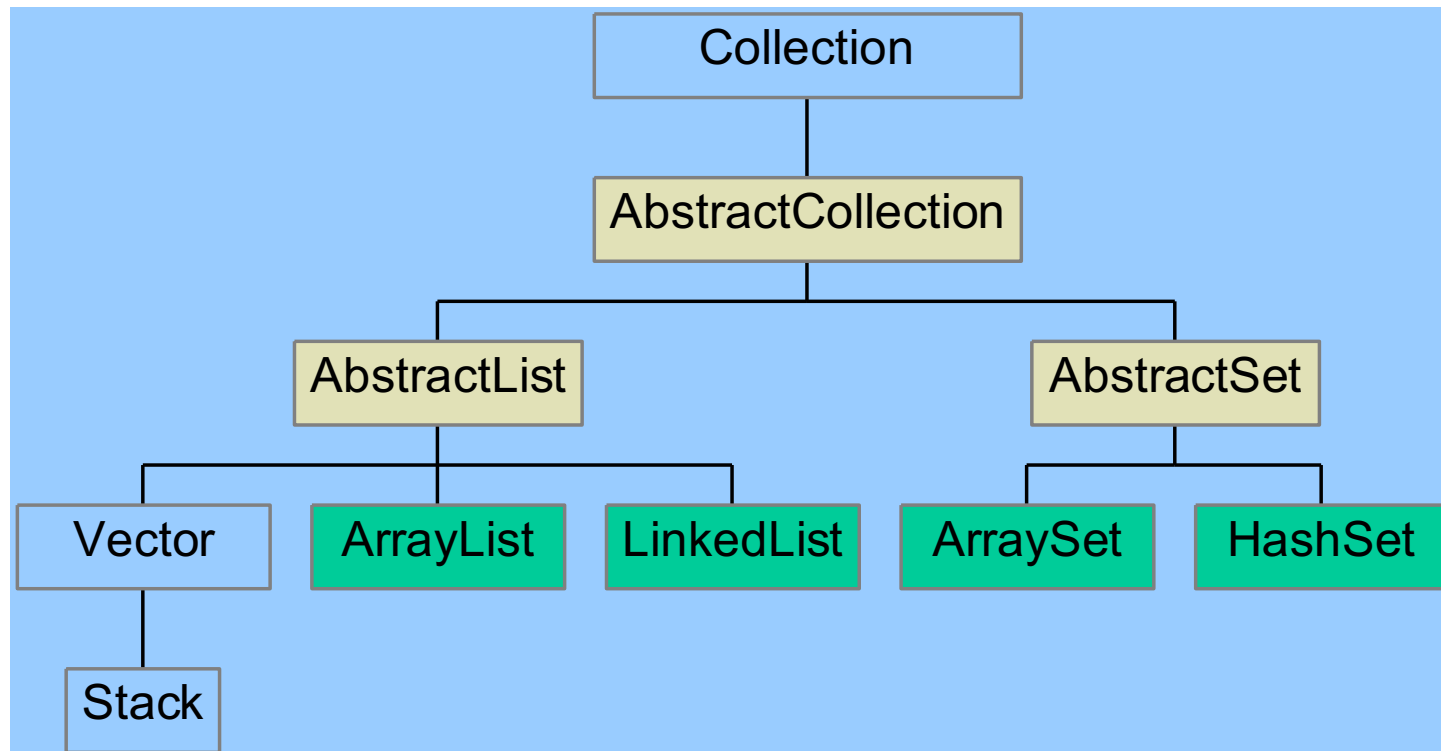
```
interface Shape1
{
    default void area()
    {
        System.out.println("Shape-1 method");
    }
}

interface Shape2
{
    default void area()
    {
        System.out.println("Shape-2 method");
    }
}

class Circle implements Shape1, Shape2
{
    //error, cannot use duplicate default methods from multiple interfaces
}
```

Reusable polymorphic code

java.util.Collection



java.util.Collection

- boolean add(Object)
- boolean addAll(Collection)
- void clear()
- boolean contains(Object)
- boolean containsAll(Collection)
- boolean isEmpty()
- Iterator iterator()
- boolean remove(Object)
- boolean removeAll(Collection)
- boolean retainAll(Collection)
- int size()
- Object[] toArray()

Iterators

- boolean hasNext()
- Object next()
- void remove()
- Por ejemplo:

```
public void print (Collection c) {  
    for (Iterator it=c.iterator();  
it.hasNext();) {  
        System.out.println(it.next());  
    }  
}
```

Collections sample

```
public static void main( String args[] )
{
    Collection<String> c;

    c = new ArrayList<String>();
    test(c);
    c = new LinkedList<String>();
    test(c);
    c = new HashSet<String>();
    test(c);
    c = new TreeSet<String>();
    test(c);
}

public static void test(Collection<String> c)
{
    String strings[] = {"uno", "dos", "tres", "cuatro", "tres"};

    for(String num:strings)
        c.add(num);

    for(String num:c)
        System.out.println(num);

    System.out.println( "-----");
}
}
```

Comparator

`java.util.Comparator<T>`

`int compare(T o1, T o2)`

Compares its two arguments for order. Returns a negative integer, zero, or a positive integer as the first argument is less than, equal to, or greater than the second.

`boolean equals(Object obj)`

Indicates whether some other object is "equal to" this Comparator.

`java.util.Collections`

`static void sort (List<T> list, Comparator<T> c)`

Sorts the specified list according to the order induced by the specified comparator.

Example

```
import java.util.Comparator;

public class NameComparator implements Comparator<Client> {

    public int compare(Client p1, Client p2) {

        String s1 = p1.getName();
        String s2 = p2.getName();

        return (s1.compareTo(s2));

    }

}

public static void main(String args[]) {
    Client p1 = new Client( name: "jose", age: 30);
    Client p2 = new Client( name: "pedro", age: 24);
    Client p3 = new Client( name: "jcarlos", age: 100);

    List<Client> c = new ArrayList<>();
    c.add(p1);
    c.add(p2);
    c.add(p3);

    System.out.println("-No ordering-----");
    print(c);

    NameComparator t1 = new NameComparator();
    Collections.sort(c, t1);
    System.out.println("-Ordered by name-----");
    print(c);

    AgeComparator t2 = new AgeComparator();
    Collections.sort(c, t2);
    System.out.println("-Ordered by age-----");
    print(c);

}
}
```

```
import java.util.Comparator;

public class AgeComparator implements Comparator<Client> {

    public int compare(Client p1, Client p2) {

        int s1 = p1.getAge();
        int s2 = p2.getAge();

        if (s1 > s2)
            return 1;
        else if (s1 < s2)
            return -1;
        else
            return 0;

    }

}
```

```
import java.util.*;

public class Test3 {

    public static void print(Collection c )
    {
        Iterator it = c.iterator();
        while( it.hasNext() )
        {
            Persona p = (Persona)it.next();
            System.out.println( "nombre:" + p.nombre + " edad:" + p.edad );
        }
        System.out.println( "-----" );
    }

    public static void main( String args[] ) {
        Persona p1 = new Persona ( "jose", 23 );
        Persona p2 = new Persona ( "pedro", 24 );
        Persona p3 = new Persona ( "jcarloe", 100 );
        List c = c = new ArrayList();
        c.add(p1);
        c.add(p2);
        c.add(p3);
        System.out.println( "-Sin ordenar-----" );

        print( c );

        per1 t1 = new per1();

        System.out.println( "-Ordenada por nombre-----" );

        Collections.sort( c, t1 );
        print( c );

        System.out.println( "-Ordenada por edad-----" );

        per2 t2 = new per2();
        Collections.sort( c, t2 );
        print( c );
    }
}
```

Parametric Polymorphism: Generics

- Parametric types
 - Collections: `List<Animal>`
- Compiler ensures type correctness in compile time, avoiding run time `ClassCastException` !
- Improves robustness and legibility

Example

- Before:

```
List farm = new ArrayList();  
farm.add(perro);  
Animal a = (Animal)farm.get(0);  
farm.add("hola");  
Animal a = (Animal)farm.get(1);  
// Runtime ERROR !!!
```

- After:

```
List<Animal> farm = new ArrayList<Animal>();  
farm.add(perro);  
Animal a = farm.get(0); // No casting  
farm.add("hola");      // Compile time ERROR !!!
```

Only saves castings ?

- Compile time checks
 - Avoids ClassCastException in runtime
- Improves legibility
 - Now you read the code and can be aware of the types contained in data structures.

Generic class

```
class Thing<T> {  
    T name;  
    public Thing(T name) {  
        this.name = name;  
    }  
    public void setName(T name) {  
        this.name = name;  
    }  
    public T getName() {  
        return name;  
    }  
}
```

Using a generic class

```
Thing<String> athing = new  
Thing<String>("pedro");  
String nombre = cosa.getNombre();  
cosa.setNombre(new Integer(1)); // ERROR !!!
```

Type Erasure

- Unlike C++, the compiler removes or erases generic type information to maintain compatibility in bytecodes.
- Parametric polymorphism only exists in compilation time. Java converts to casting and Objects in the generated bytecode (castings, $T \rightarrow \text{Object}$, ...).
- Instances do not contain information about generic types in run time. Everything is an Object.
- `List<Integer>` and `List<Thing>` are `List`.

Nested types

- It is possible:

-class Thing <T extends Animal<E>>

- Or:

```
LinkedList<LinkedList<String>> zss = new  
    LinkedList<LinkedList<String>>();
```

```
class Thing <T extends Map<Foo,Bar>>
```

Bounded Parametric Polymorphism

- Restrict types using inheritance `<T extends Animal>`
- It allows us to use operations from the bounded types inside our (not so) generic code.

Example:

```
class Thing2<T extends Animal> {  
    T nombre;  
    public Thing2(T name) {  
        this.name = name;  
    }  
    public void setName(T name) {  
        this.nombre = nombre;  
    }  
    public T getName() {  
        return name;  
    }  
    public void saluda(){  
        nombre.habla();  
    }  
}
```

Example

```
Thing2<String> aThing = new Thing2<String>("pedro");  
// FALLO !!!  
Thing3<Perro> other = new Thing2<Perro>(new  
Perro());  
Perro p = cosa.getName();  
Animal a = cosa.getName();  
System.out.println(p.habla());  
System.out.println(a.habla());
```

Interface Collection<E>

boolean **add**(E o)

boolean **addAll**(Collection<? extends E> c)

void **clear**()

boolean **contains**(Object o)

boolean **containsAll**(Collection<?> c)

boolean **equals**(Object o)

int **hashCode**()

boolean **isEmpty**()

Iterator<E> **iterator**()

boolean **remove**(Object o)

boolean **removeAll**(Collection<?> c)

boolean **retainAll**(Collection<?> c)

Interface Iterator<E>

boolean hasNext()

E next()

void remove()

Interface Map<K,V>

void clear()

boolean containsKey(Object key)

boolean containsValue(Object value)

Set<Map.Entry<K,V>> entrySet()

boolean equals(Object o)

V get(Object key)

int hashCode()

boolean isEmpty()

Set<K> keySet()

V put(K key, V value)

void putAll(Map<? extends K,? extends V> t)

V remove(Object key)

int size()

Collection<V> values()

java.util Interface Comparator<T>

int compare(T o1, T o2)

java.util Class Collections

static <T extends Comparable<? super T>> void sort(List<T> list)
<T> void sort(List<T> list, Comparator<? super T> c)

static <T> int binarySearch(List<? extends Comparable<? super T>> list, T key)

static <T> int binarySearch(List<? extends T> list, T key,
Comparator<? super T> c)

Examples

```
public interface Imap<T> {  
    public T aplica(T elem);  
}  
  
public class Increment implements Imap<Integer> {  
    public Integer aplica(Integer elem) {  
        return elem+1;  
    }  
}  
  
public class Cuadrado implements Imap<Integer>{  
    public Integer aplica(Integer elem) {  
        return elem*elem;  
    }  
}
```

```
public class Mayusculas implements Imap<String> {  
    public String aplica(String elem) {  
        return elem.toUpperCase();  
    }  
}
```

```
public class Map {  
    public static <T> List<T> map(List<T> lista,  
                                   Imap<T> funcion){  
        List<T> result = new LinkedList<T>();  
        for (T elem: lista)  
            result.add(funcion.aplica(elem));  
        return result;  
    }  
}
```

```
int lista[] = {1,2,3,4,5};  
String lista2[] =  
{"pedro","lenguajes","programacion","Java","haskell"};
```

```
LinkedList<Integer> listaint = new LinkedList<Integer>();  
for (int i:lista)  
listaint.add(i);  
LinkedList<String> listastring = new LinkedList<String>();  
for (String i:lista2)  
listastring.add(i);
```

```
List<Integer> result = Map.map(listaint,new Increment());  
System.out.println("lista de incrementos:");  
for (Integer elem:result)  
System.out.println(elem);
```

Compared to 1.4 ¿?

```
System.out.println("lista de cuadrados:");  
List<Integer> result2 = Map.map(listaint,new Cuadrado());  
for (Integer elem:result2)  
System.out.println(elem);
```

```
System.out.println("lista de cadenas en mayuscula:");  
List<String> result3 = Map.map(listastring,new Mayusculas());  
for (String elem:result3)  
System.out.println(elem);
```

for iterator

- Syntax:

`for(type var : array) {...}`

or `for(type var : collection) {...}`

- Example (Array):

```
for(float x : myArray)
    res += x;
```

- Example (Collections)

Before:

```
for (Iterator iter = farm.iterator(); iter.hasNext(); )
    ((Animal) iter.next()).talk();
```

Now:

```
for (Animal a : farm)
    a.talk();
```

Iterating in your classes

java.lang interface Iterable<T>

Iterator<T> iterator()

Returns an iterator over a set of elements of type T.

java.util interface Iterator<E>

boolean hasNext()

Returns true if the iteration has more elements.

E next()

Returns the next element in the iteration.

void remove()

Removes from the underlying collection the last element returned by the iterator (optional operation).

Closures: Anonymous functions in Java

```
public class Map2 {  
  
    public static <T> List<T> map(List<T> list, Function<T,T> f) {  
        List<T> result = new LinkedList<T>();  
        for (T elem:list){  
            result.add(f.apply(elem));  
        }  
        return result;  
    }  
  
    public static void main(String args[]){  
        List<String> elems = new LinkedList<>();  
        elems.add("tap");  
        elems.add("lp");  
        List <String> result = Map2.map(elems, (String p) -> p.toUpperCase());  
        for (String elem:result)  
            System.out.println(elem);  
    }  
}
```

Lambdas in Java

A lambda converts to **an instance of a class that implements a single method interface**.

A lambda consists of a parameter list followed by the arrow token and a body, as in:

- *(parameterList) -> {statements}*

For example, the following lambda receives two `ints` and returns their sum:

- `(int x, int y) -> {return x + y;}`

This lambda's body is a statement block that may contain one or more statements enclosed in curly braces.

A lambda's parameter types may be omitted, as in:

- `(x, y) -> {return x + y;}`

in which case, the parameter and return types are determined by the lambda's context.

Functional Interfaces

@FunctionalInterface
public interface **Function**<T,R>

Represents a function that accepts one argument and produces a result.

This is a functional interface whose functional method is `apply(Object)`.

Since:

1.8

Method Summary

All Methods	Static Methods	Instance Methods	Abstract Methods	Default Methods
Modifier and Type		Method and Description		
default	<V> Function <T,V>	andThen (Function <? super R,? extends V> after) Returns a composed function that first applies this function to its input, and then applies the after function to the result.		
	R	apply (T t) Applies this function to the given argument.		
default	<V> Function <V,R>	compose (Function <? super V,? extends T> before) Returns a composed function that first applies the before function to its input, and then applies this function to the result.		
static	<T> Function <T,T>	identity () Returns a function that always returns its input argument.		

@FunctionalInterface
public interface **Consumer**<T>

Represents an operation that accepts a single input argument and returns no result. Unlike most other functional interfaces, Consumer is expected to operate via side-effects.

This is a functional interface whose functional method is `accept(Object)`.

Since:

1.8

Method Summary

All Methods	Instance Methods	Abstract Methods	Default Methods
Modifier and Type		Method and Description	
	void	accept (T t) Performs this operation on the given argument.	
default	Consumer <T>	andThen (Consumer <? super T> after) Returns a composed Consumer that performs, in sequence, this operation followed by the after operation.	

Conclusions

- Concepts
 - Understand key OOP concepts in Java
 - Polymorphism in Java
 - Code reuse and polymorphism (contract, polymorphic assignment, dynamic binding, generic code)
 - Multiple Inheritance and Interfaces
- Check your knowledge in Kahoot
 - TAP2 Object Oriented Programming
 - https://kahoot.it/challenge/08043650?challenge-id=7fef0780-bb84-496e-a4e6-379221fcc532_1600939847073
- This week TAP2
 - <https://github.com/pedrotgn/TAP/tree/master/TAP2>